Professorship of Energy Management Technologies
TUM School of Engineering and Design
Technical University of Munich

Energy Informatics Lab
Prof. Dr. Christoph Goebel

Assignment 4: Optimal Power Flow

# Meeting	Location	Content
1st Session	N3815	Introduction of assignment; work on tasks
Until 1st Session of next assignment	N3815	Work on task; presentation
Until 2nd Session of next assignment	Home	Work on report

Background

Optimal Power Flow is an optimization problem focused on determining the best operating levels for electric power plants to meet demands, while simultaneously trying to minimize the operating cost of said power generation. These electric power flows are limited by constraints of the system's physical characteristics, such as power balances at buses, network operating limits (in terms of line flows and voltages), as well as limits on control variables. As such, this is not only a complex, but also an important problem, since this optimization often needs to be solved very frequently, and in real time, in order to ensure demand is met accurately.

This a classic method for alternative current networks; however, under certain assumptions, the power flow equations can be linearized, leading to the so-called DC-OPF.

For this exercise, we will be solving both the AC and DC Optimal Power Flow problems for a given transmission network. In order to do this, we will be leveraging the software package *pyomo*, which is a Python-based open-source package for modeling structured optimization problems.

Before we do this, we will so an introductory exercise to get used to using pyomo. To run the optimizations in pyomo, we will be using the NEOS server, which is a free internet-based service that provides different solvers for solving numerical optimization problems. To use NEOS in your notebooks, you will need to use a valid TUM email address and include it in your code, as indicated in NEOS' Pyomo Guide.

Additionally, you will need a python environment with the following packages:

- jupyter
- numpy
- pandas
- pyomo
- openpyxl

Energy Informatics Lab
Prof. Dr. Christoph Goebel

The aim of the assignment is to evaluate:

- your understanding of how power system constraints can be modeled,
- your ability to rigorously formulate optimization problems,
- your ability to apply the pyomo framework on your task.

The expected outcome of the assignment includes:

- a report of maximum 2 pages (excluding appendices),
- your ipython notebook (.ipynb file), as well as the optimization results as .json files.

The following aspects of your work will be graded:

- Formulation of optimization problem (correctness)
- Ability to work with pyomo
- Evaluation and interpretation of optimal solution
- Discussion (what went well, what went wrong, how the forecasts could be improved - in your opinion, etc.)
- General presentation of the assignment report (flow of information, quality of English, quality of illustrations, etc.)

Stage 0 – First Optimization Problem

To get familiar with the pyomo optimization framework, we will first work on a simple dummy problem. For given $a_1, a_2, c \in R^n$, we want to minimize

$$\sum_{i=1}^{n} a_{2,i} x_i^2 + a_{1,i} x_i$$

s.t.

$$x_i \geq c_i \quad \forall i = 1,\dots,n$$

over $x \in R^n$. $a_1, a_2, c \in R^n$ are given as **DataA.csv** in moodle.

Before we start solving this

To solve this problem in pyomo, please follow these steps:

1) Define the objective and constraint as python functions. The objective takes the argument “model”, which later will store all required parameters and variables. The constraint takes the argument “model” and the index i.
2) Now model the problem in pyomo. Initialize the model as a “concrete model” (since parameters are known) and set
   - the dimension n as “Set”
   - the optimization variable x as “Var”
   - the objective function as “Objective” and
   - the constraint as “Constraint”.

Moreover, assign the parameters to the model, via defining them as attributes of your model object.

3) Use the NEOS server to optimize your model.
4) How did you do? Please look at
   - the status of your optimization (should be “ok”),
   - the optimal x-value (should be (0, 1, 0, -0.75, 0, -2, -6.5)’)
   - and the resulting minimum (should be -49.375).

Check out the pyomo docs for more help and examples of how to use pyomo.

Stage 1 – Alternating Current OPF

Introduction

Now that you are a bit more comfortable with Pyomo, we can start taking a look at the AC Optimal Power Flow (ACOPF) problem. It can be complicated at first glance, but we will walk you through it. In the general case, this problem is formulated as such:

$$min \sum_{i \in (G)} C_i\left(P_i^G\right) \tag{1.1}$$

s.t.

$$P_i^G - P_i^L - P_i(V,\delta) = 0 \quad \forall i \in N \tag{1.2}$$
$$Q_i^G - Q_i^L - Q_i(V,\delta) = 0 \quad \forall i \in N \tag{1.3}$$
$$P_i^{G,min} \le P_i^G \le P_i^{G,max} \quad \forall i \in G \tag{1.4}$$
$$Q_i^{G,min} \le Q_i^G \le Q_i^{G,max} \quad \forall i \in G \tag{1.5}$$
$$V_i^{min} \le V_i \le V_i^{max} \quad \forall i \in N \tag{1.6}$$
$$\delta_i^{min} \le \delta_i \le \delta_i^{max} \quad \forall i \in N \tag{1.7}$$

with the AC power flow equations for each node i in N:

$$P_i(V,\delta) = V_i \sum_{k=i}^{N} V_k\left(G_{i,k} cos\left(\delta_i - \delta_k\right) + B_{i,k} sin\left(\delta_i - \delta_k\right)\right) \tag{1.8}$$

$$Q_i(V,\delta) = V_i \sum_{k=i}^{N} V_k\left(G_{i,k} sin\left(\delta_i - \delta_k\right) - B_{i,k} cos\left(\delta_i - \delta_k\right)\right) \tag{1.9}$$

where the voltages are defined in polar coordinates, $\widehat{V_i} = V_i \angle \delta_i$, and the admittance in rectangular coordinates $Y_{i,k} = G_{i,k} + jB_{i,k}$.

Additionally, the cost function for a generator i is defined as follows:

$$C_i\left(P_i^G\right) = a_i\left(P_i^G\right)^2 + b_i P_i^G + c_i$$

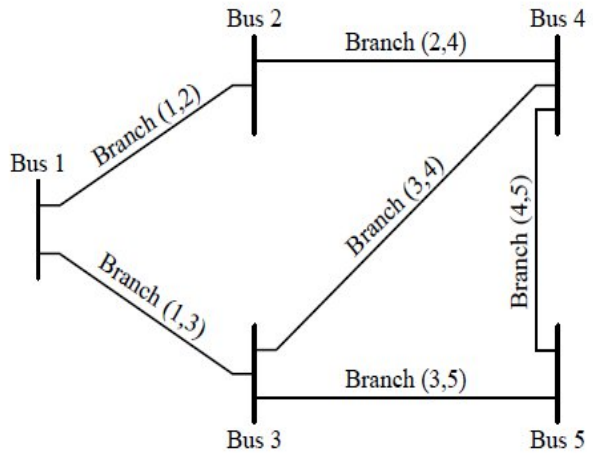
where a_i, b_i, and c_i are generator cost coefficients.

Note: Additional variable definitions can be found in the Appendix.

Task

We will solve the AC-OPF problem for a 5 bus system, similar to the reference figure below, using NEOS. The starter code for the implementation is provided in *opf_starter.ipynb* and the parameter data in *OPFData.xlsx*. You should also have the *opf_utils.py* file located in the same directory as the *opf_starter.ipynb* file. All files are available for download via Moodle.

Note: that all values are given in p.u. (per unit) and the table data values below may not always be consistent with the provided data file.



From Bus i	To Bus k	Series Resistance R_{ik}	Series Reactance X_{ik}	Shunt Susceptance b_{ik}^{Sh}	Voltage Ratio T_{ik}	Phase Angle φ_{ik}
1	2	0.000	0.300	0.000	·	·
1	3	0.023	0.145	0.040	·	·
2	4	0.006	0.032	0.010	·	·
3	4	0.020	0.260	0.000	·	-3.0°
3	5	0.000	0.320	0.000	0.98	·
4	5	0.000	0.500	0.000	·	·

Bus i	Min. Generator Real Power $P_i^{\mathrm{G,min}}$	Max. Generator Real Power $P_i^{\mathrm{G,max}}$	Min. Generator Reactive Power $Q_i^{\mathrm{G,min}}$	Max. Generator Reactive Power $Q_i^{\mathrm{G,max}}$
1	$-\infty$	∞	$-\infty$	∞
3	0.10	0.40	-0.20	0.30
4	0.05	0.40	-0.20	0.20

Bus i	Load Real Power P_i^{L}	Load Reactive Power Q_i^{L}	Min. Bus Voltage $V_i^{\min}$	Max. Bus Voltage $V_i^{\max}$
1	·	·	1.00	1.00
2	·	·	0.95	1.05
3	·	·	0.95	1.05
4	0.900	0.400	0.95	1.05
5	0.239	0.129	0.95	1.05

To set up the optimization problem, the correct data needs to be loaded into the model. We will use the provided data files, and implement the objective function and the linear and non-linear constraints. Please follow the starter code to:

1) Add implementations for data loading
2) Define the objective function
3) Define the linear and non-linear constraints

All model definitions are based on equations (1.1) - (1.9). You are responsible for filling out every location with “TODO” under the **Model Setup** and the **AC Optimal Power Flow Setup** sections.

Hint: It would be helpful to go through the **Parameter Definitions** section in order to make sure you have a good understanding of the data structure and its interpretation. Additionally, some variable names that you will need are defined in the *load_model* function.

Upload

After this is correctly implemented, you should be able to run the **Load Model from Data** cell and then the **Solve AC Model** cell to perform your optimization and print the results for the AC model.

The solver throws an error that the model is infeasible. Ignore this error and provide the answers.

Please upload your **ipython notebook** and your **optimization results** in the form of a .json file to moodle, named in the format below (section "Naming of Submissions"). For this we have provided some helper functions in *opf_utils.py*. Ensure the json file has the following form:

```
{
    'name': your team's name as string,
    'task': 'AC',
    'cost': the optimal cost from the optimization
    'nodes': [
        {
            'id': the node number, starting from '1'
            'voltage_angle': the node's optimal voltage angle
            'voltage_magnitude': the node's optimal voltage magnitude
            'power': the node's optimal power
            'reactive_power': the node's optimal reactive power
        },
        ...,
        {
            'id': see above
            'voltage_angle': see above
            'voltage_magnitude': see above
            'power': see above
            'reactive_power': see above
```

```
            },
        ]
}
```

Stage 2 – Direct Current OPF

Introduction

The relaxed version of the AC-OPF problem is called DC-OPF. The relaxation assumes that the transmission system is lossless, $G_{i,k} = 0 \; \forall \; i,k$. Also, it is assumed that the difference between the adjacent bus voltage angles is small, such that $\sin(\delta_i - \delta_k) \approx \delta_i - \delta_k$ and $\cos(\delta_i - \delta_k) \approx 1$. Moreover, reactive power is neglected, which is a reasonable assumption when we do not have too many inductive and capacitive loads in our system. Lastly, the voltage magnitude of all nodes is assumed as 1.0 p.u., since we do such a good job at managing this reactive power.

With these assumptions the power flow equations in (1.8) and (1.9) reduce to:

$$P_i(\delta) = \sum_{k=i}^{N} B_{i,k}\left(\delta_i - \delta_j\right) \tag{1.10}$$

The DC-OPF problem can then be written as follows:

$$min \sum_{i \in (G)} C_i\left(P_i^G\right) \tag{1.11}$$

s.t.

$$P_i^G - P_i^L - P_i(\delta) = 0 \tag{1.12}$$

$$P_i^{G,min} \leq P_i^G \leq P_i^{G,max} \quad \forall i \in G \tag{1.13}$$

$$\delta_i^{min} \leq \delta_i \leq \delta_i^{max} \quad \forall i \in N \tag{1.14}$$

Task

We are going to adapt our AC-OPF solution to fit the DC case. To do this, we will need to adjust our constraints. Please continue to build on top of the starter to adjust the necessary constraints, based on equations (1.10) - (1.14). You are responsible for filling out every location with “TODO” under the **DC Optimal Power Flow Setup** section.

Upload

After this is correctly implemented, you should be able to run the **Load Model from Data** cell again and then the **Solve DC Model** cell to perform your optimization and print the results for the DC model.

Please upload your **ipython notebook** (in total one notebook with AC and DC code together) and your **optimization results** in the form of a .json file to moodle, named in the format below (section "Naming of Submissions"). For this we have provided some helper functions in *opf_utils.py*. Ensure the json file has the following form:

```
{
    'name': your team's name as string,
    'task': 'DC',
    'cost': the optimal cost from the optimization,
    'nodes': [
        {
            'id': the node number, starting from '1'
            'voltage_angle': the node's optimal voltage angle
            'power': the node's optimal power
        },
        ...,
        {
            'id': see above
            'voltage_angle': see above
            'power': see above
        },
    ]
}
```

You're all done!

Make sure to submit all required files on Moodle by the appropriate deadline. Only one submission per group necessary.

Naming of Submissions

Each submission must be named according to the following pattern:

- Presentation: f"Presentation_4_Group_{Group_letter}.pdf“
- Report and Code: f"Report_Code_4_Group_{Group_letter}.zip“
- Results AC: f"Results_4_AC_Group_{Group_letter}.json"
- Results DC: f"Results_4_DC_Group_{Group_letter}.json"

As an example for the group submissions of Group E:

- “Presentation_4_Group_E.pdf”
- "Report_Code_4_Group_E.zip“
- "Results_4_AC_Group_E.json"
- "Results_4_DC_Group_E.json"

Appendix: Short description of data and various files

Variable Definitions

N – the set of all nodes

G – the set of generators

L – the set of loads

P_i^G – the real power of generator G at node i

P_i^L – the real power drawn by loads at node i

Q_i^G – the reactive power of generator G at node i

Q_i^L – the reactive power drawn by loads at node i

V_i– the voltage magnitude at node i

δ_i– the voltage angle at node i

$G_{i,k}$ – the real part of the admittance between nodes i and k

$B_{i,k}$ – the imaginary part of the admittance between nodes i and k

$C_i\left(P_i^G\right)$ – the cost function of power generation at node i

a_i, b_i, c_i – generator cost coefficients for node i

opf_starter.ipynb

ipython notebook file to be used as a starting point to solve this lab exercise. This file contains helpful parameter descriptions, function definitions, and overall code structure to be able to successfully complete the assignment.

opf_utils.py

This is a utility python file to provide support for solving and printing results of the solved optimization problem.

OPFData.xlsx

Excel file containing all parameters needed to set up the OPF optimization problem. The excel file consists of three sheets: NodeData, RealAdmittanceMatrix, and ImaginaryAdmittanceMatrix.

The columns for NodeData are all clearly marked in the sheet. The RealAdmittanceMatrix and ImaginaryAdmittanceMatrix sheets have no columns and can be used to load into the appropriate variables.